`FaultBasis` — the fault-local coordinate frame

Developer walkthrough of `fault/fault_basis.hpp`

SEAS-MFEM · SAFS dynamic rupture

2026-05-23

Contents

1 Overview

`FaultBasis` is the **geometric bridge** between the bulk elasticity solver and the friction law. The DG elasticity operator lives in global Cartesian coordinates (x, y, z); it produces a traction vector and consumes a slip vector, both expressed in that global frame. The friction law (rate-and-state or slip-weakening) instead speaks in **fault-local** quantities: a scalar normal stress σ_n and tangential traction components resolved onto the **dip** and **strike** directions of the fault surface.

`FaultBasis` precomputes, for every fault face — and, when accuracy on a curved fault demands it, for every quadrature point on that face — an orthonormal frame

$$\{\, \hat{\mathbf{n}}\ (\text{normal}),\quad \mathbf{t}_1\ (\text{dip}),\quad \mathbf{t}_2\ (\text{strike}) \,\}$$

and exposes four projection helpers that move data between the global and local frames:

Direction	Helper	What it computes
global → local	`ProjectTraction`	$\tau_1 = \mathbf{T}\cdot\mathbf{t}_1,\ \tau_2 = \mathbf{T}\cdot\mathbf{t}_2$
global → local	`NormalStress`	$\sigma_n = -\,\mathbf{T}\cdot\hat{\mathbf{n}}$
local → global	`EmbedSlip` / `EmbedSlipQP`	$\Delta\mathbf{u} = s_1\mathbf{t}_1 + s_2\mathbf{t}_2$

The class is **header-only** (`fault/fault_basis.hpp`), all methods inline. Its single reason for existing is to give the friction coupling **one** fault-local frame that is computed the same way everywhere — on every face, at every quadrature point, and on both MPI ranks of a shared face — and that stays well-defined on the non-planar SAFS fault, where a naive orientation rule is ambiguous.

> **One-sentence summary.** `FaultBasis` turns the raw face normal that MFEM hands it into a signed, orthonormal `(normal, dip, strike)` frame — one that two MPI ranks sharing a face are guaranteed to agree on up to a global sign — and then does the dot products that resolve tractions into, and embed slip out of, that frame.

---

2 Coordinate conventions you must hold in your head

Everything below assumes the **reference frame this code adopts**, fixed once at the call site (`elasticity_operator_setup.inl:728–737`, `wave_operator.inl:339–349`):

```
Vector ref_normal(3); ref_normal = 0.0; ref_normal(1) = -1.0;  // (0, -1, 0)
Vector up(3);         up = 0.0;         up(2) = 1.0;           // (0,  0, 1)
```

- **Axes.** X = along-strike, Y = fault-normal, Z = depth (negative downward, $Z < 0$ is into the earth). The canonical planar fault sits at $Y = 0$.

- **`ref_normal`** $= (0, -1, 0)$ — the reference direction used to *orient* each computed normal consistently. For a planar $Y = 0$ fault this *is* the fault normal; on a curved fault it is only a tie-breaker (see §6).
- **`up`** $= (0, 0, 1)$ — the reference "up" used to split the tangent plane into dip and strike. It must never be collinear with the face normal.
- **`tangent1` = dip, `tangent2` = strike.** This is the project-wide convention (BP5 / TPV102); in `DOFData`, components `V1/slip1/tau1` are dip and `V2/slip2/tau2` are strike.

These two vectors are **inputs** to `FaultBasis` — the class itself hardcodes nothing. The SEAS operators pass the values above; a different problem could pass different ones.

---

3 The signed-vector contract and `sign_flipped`

This is the single most important convention in the class, so it gets its own section.

3.1 The problem: the raw normal fixes a *line*, not an *arrow*

The only geometric input we get for a face is the raw normal that MFEM's `CalcOrtho` builds from the face Jacobian. **Its direction is not unique.** It depends on how that particular face transformation was set up — which of the two elements sharing the face is treated as "element 1," the local vertex ordering, and, in parallel, which rank is asking. For one and the same physical fault surface, two evaluation contexts can hand us normals that point in **opposite** directions. The raw normal pins down the *line* the normal lives on, but not which way the *arrow* points.

That is a problem, because the friction law needs a **single-valued** frame: dip and strike must mean the same physical directions at every point on the fault and on every rank. If the arrow were allowed to flip from place to place, the slip we embed on one side and the traction we read on the other would be expressed in inconsistent frames, and the coupling would be silently wrong.

3.2 The fix: canonicalize, and remember whether you flipped

So we **choose** a canonical arrow and force every raw normal onto it. The rule (§6) is "point the normal so that $\mathbf{n} \cdot$ `ref_normal` > 0," with a well-defined fallback for the case where the normal is nearly perpendicular to `ref_normal`.

`sign_flipped` is the boolean that records **whether the incoming raw normal had to be negated to reach that canonical arrow.** `ComputeOrientedFrame` uses it in two ways:

1. **During construction** it flips the raw normal to canonical, builds the dip/strike frame from the canonical normal, then negates the *entire* frame back if `sign_flipped` was true. The net effect is that the **stored** frame is $($ `sign_flipped` $? -1 : +1) \times$ canonical — a deterministic function of the physical normal *line* and the rule, **not** of which element or rank happened to seed the raw normal.
2. **As a stored breadcrumb** so a downstream consumer can reconstruct the *canonical* (pre-negation) frame from the stored one — which is exactly what the dynamic-rupture flux does to get a side-independent answer (see §5.2).

3.3 Why bake the sign into the vectors instead of carrying a factor

Once the frame is built, the sign is **baked into** `normal`, `tangent1`, `tangent2` — the stored vectors already point the canonical (or its negation) way. Consumers therefore do a **bare dot product** with the stored tangent; there is **no $\pm$ `sign` factor anywhere** in `ProjectTraction`, `NormalStress`, or `EmbedSlip`.

This is a deliberate design choice with a concrete payoff. Slip is *embedded* with the very same signed tangent that traction is *projected* onto, so whatever sign sits in that tangent **cancels** across the slip $\rightarrow$ traction $\rightarrow$ friction $\rightarrow$ slip round trip. There are no scattered sign factors at the call sites to get wrong. Re-applying a sign on top of the stored vectors would double-flip and break that cancellation — the classic positive-feedback blow-up (§ Gotchas).

> **Mental model for `sign_flipped`.** It is not a property of the *fault* — it is a property of the *raw normal we happened to be handed*. Two contexts looking at the same face can disagree on `sign_flipped`; what they must never disagree on is the **canonical** frame, which is `sign_flipped`-corrected and therefore identical for both.

---

4 Data structures

```
struct FaultBasisQPData {     // per quadrature point
   real_t normal[3];          // unit normal (sign baked in)
   real_t tangent1[3];        // dip          (sign baked in)
   real_t tangent2[3];        // strike       (sign baked in)
   real_t nl;                 // |n_raw| at this QP (pre-normalization length)
   bool   sign_flipped;       // orientation breadcrumb (see §3)
};

struct FaultBasisData {       // per face
   real_t normal[3];          // centroid frame (sign baked in)
   real_t tangent1[3];
   real_t tangent2[3];
   bool   sign_flipped;
   std::vector<FaultBasisQPData> qp_data;   // per-QP frames
};
```

`FaultBasisData` carries **two** representations of the same frame:

1. **A centroid frame** (`normal`, `tangent1`, `tangent2`) — one frame per face, evaluated at the face centre. Fast, and exact for a planar face where the normal is constant.
2. **A per-QP frame** (`qp_data`) — one frame per quadrature point. This is the accurate representation on a *curved* face, where the normal sweeps across the facet and a single centroid frame is no longer exact.

`nl` is the **un-normalized** length $|J_F|$ of the raw normal that `CalcOrtho` returned — the face-Jacobian determinant, i.e. the local face-measure weight. We capture it before normalizing, both to guard

against degenerate faces (`MFEM_VERIFY(nl > 0)`) and as a per-QP area weight. `sign_flipped` carries the orientation breadcrumb from §3: the sign is already inside the basis vectors, so for the *vectors* it is informational — but the dynamic-rupture flux in `wave_operator.inl` reads it to rebuild the *canonical* (pre-negation) frame, so it must be identical on both ranks of a shared face.

---

5 The core algorithm: `ComputeOrientedFrame`

This static method is the heart of the class. Given a raw face normal it produces the signed `(normal, dip, strike)` frame. It is `static` and pure (no class state), which is why `test_fault_basis_dip_strike_symmetry.` can hammer it with 1000 perturbed normals without building a mesh.

```
static void ComputeOrientedFrame(Vector &n_raw, int dim,
                                 const Vector &ref_normal, const Vector &up,
                                 real_t normal[3], real_t tangent1[3],
                                 real_t tangent2[3], bool &sign_flipped,
                                 real_t &nl);
```

- **Inputs:** `n_raw` (raw face normal from `CalcOrtho`, **modified in place**), `dim` (2 or 3), `ref_normal`, `up`.
- **Outputs:** `normal`, `tangent1`, `tangent2` (each 3 components, the last zero in 2D), `sign_flipped`, `nl`.

5.1 The five construction steps

```
nl = n_raw.Norml2();                                          // (1)

sign_flipped = NormalNeedsFlipToCanonical(n_raw, ref_normal, dim);  // (2)

if (sign_flipped) { n_raw.Neg(); }                            // (3)
if (nl > 0.0)     { n_raw /= nl; }                            // (4) normalize

// ... zero the outputs, then normal[d] = n_raw(d) ...

if (dim == 3)                                                 // (5) tangent frame
{
   // strike = normalize(up × n)
   s[0] = up(1)*n_raw(2) - up(2)*n_raw(1);
   s[1] = up(2)*n_raw(0) - up(0)*n_raw(2);
   s[2] = up(0)*n_raw(1) - up(1)*n_raw(0);
   // ... s /= |s|  (MFEM_VERIFY |s| > 1e-12: up not collinear with n) ...

   // dip = strike × n
   dv[0] = s[1]*n_raw(2) - s[2]*n_raw(1);
```

```
   dv[1] = s[2]*n_raw(0) - s[0]*n_raw(2);
   dv[2] = s[0]*n_raw(1) - s[1]*n_raw(0);
   // ... dv /= |dv|  (R-003 explicit normalize, see below) ...

   tangent1[d] = dv[d];   // dip
   tangent2[d] = s[d];    // strike
}
else // dim == 2
{
   real_t cross = up(0)*n_raw(1) - up(1)*n_raw(0);
   real_t sv = (cross >= 0.0) ? 1.0 : -1.0;
   tangent1[0] = -sv * n_raw(1);
   tangent1[1] =  sv * n_raw(0);                          // 90° rotation of n
}

if (sign_flipped)                                         // (6) negate all
{
   for (int d = 0; d < 3; d++)
   { normal[d] = -normal[d]; tangent1[d] = -tangent1[d]; tangent2[d] = -tangent2[d]; }
}
```

Step by step:

- **(1) Record `nl`.** The length of the raw normal *before* any flip or normalization. Note it is captured *before* step (3).
- **(2) Decide the sign.** `sign_flipped` says whether `n_raw` points opposite to the canonical orientation and must be negated to reach it. The decision is delegated to `NormalNeedsFlipToCanonical` (§6) — for a planar fault this reduces to the plain test $\text{sign}(\mathbf{n}_{\text{raw}} \cdot \texttt{ref\_normal})$.
- **(3) Flip to ref-aligned.** If flipped, negate `n_raw`. After this the *working* normal points the canonical way regardless of which element seeded it.
- **(4) Normalize.** Divide by `nl`. This is correct even after the `Neg()` in (3) because negation preserves length, so `|n_raw|` is still `nl`.
- **(5) Build the tangent frame** from the ref-aligned, unit normal:
  - **3D:** strike $\mathbf{t}_2 = (\mathbf{up} \times \hat{\mathbf{n}})/|\cdot|$, then dip $\mathbf{t}_1 = \mathbf{t}_2 \times \hat{\mathbf{n}}$. Because $\mathbf{t}_2 \perp \hat{\mathbf{n}}$ and both are unit, $|\mathbf{t}_1| = 1$ in exact arithmetic — but FP rounding in the strike normalize leaks in, so dip is **explicitly normalized** too (R-003, below).
  - **2D:** the single tangent is a 90° rotation of the normal, $\mathbf{t}_1 = (-n_y, n_x)$, with the sign `sv` chosen from $\text{sgn}(\mathbf{up} \times \hat{\mathbf{n}})$ so the tangent points consistently "up." It inherits unit length from the unit normal and is *not* separately normalized.
- **(6) Negate all.** If `sign_flipped`, negate `normal`, `tangent1`, `tangent2`. **This is the step that bakes the sign into the stored vectors.**

5.2 The cross-product frame, and what is orthonormal to what

In 3D the construction guarantees an orthonormal triad:

$$\mathbf{t}_2 = \frac{\mathbf{up} \times \hat{\mathbf{n}}}{|\mathbf{up} \times \hat{\mathbf{n}}|}, \qquad \mathbf{t}_1 = \mathbf{t}_2 \times \hat{\mathbf{n}}, \qquad \hat{\mathbf{n}} \cdot \mathbf{t}_1 = \hat{\mathbf{n}} \cdot \mathbf{t}_2 = \mathbf{t}_1 \cdot \mathbf{t}_2 = 0.$$

Strike is forced into the horizontal-ish plane (perpendicular to `up`); dip is then whatever completes the frame — it carries the down-dip component.

Worked example – the canonical vertical $Y = 0$ fault

Take the ref-aligned normal $\hat{\mathbf{n}} = (0, -1, 0)$ (what step (3)–(4) produce once `n_raw` is flipped to agree with `ref_normal`), with $\mathbf{up} = (0, 0, 1)$:

$$\mathbf{t}_2 = \mathbf{up} \times \hat{\mathbf{n}} = (0,0,1) \times (0,-1,0) = (1,0,0) \quad (\text{strike}, \ +X)$$
$$\mathbf{t}_1 = \mathbf{t}_2 \times \hat{\mathbf{n}} = (1,0,0) \times (0,-1,0) = (0,0,-1) \quad (\text{dip, toward depth } Z < 0)$$

These — $\mathbf{t}_1 = (0,0,-1)$, $\mathbf{t}_2 = (+1,0,0)$ — are the **canonical** frame values quoted in `CLAUDE.md` (`can_t1`, `can_t2`). They describe TPV102's pure strike-slip fault, where the strike component carries all the motion and the dip component is ≈ 0.

Canonical frame vs *stored* frame

The example above is the frame built from the *ref-aligned* normal — what step (5) produces **before** step (6). The frame `FaultBasis` actually *stores* is

$$\text{stored} = (\ \text{sign\_flipped}\ ?\ -1 : +1\ ) \times \text{canonical}.$$

Which one a given face gets depends on **which element's `CalcOrtho` seeded `n_raw`**:

- The element whose raw normal already agreed with `ref_normal` → `sign_flipped = false` → stored = canonical.
- The element on the other side → `sign_flipped = true` → stored = −canonical.

So the two sides of a face hold frames that differ by a **global sign**. The in-code comment (lines 492–502) is emphatic that this is **correct**, not a bug: the DG face normal *also* flips between the two sides ($\mathbf{n}_B = -\mathbf{n}_A$), the embedded displacement jump flips with it, and the assembly produces matching contributions (verified to `< 1e-12` by `test_serial_parallel_displacement_match`). The flux code rebuilds the canonical frame from `sign_flipped` precisely so it can undo this and recover a side-independent answer.

5.3 R-003: why dip is explicitly normalized

```
const real_t d_len = std::sqrt(dv[0]*dv[0] + dv[1]*dv[1] + dv[2]*dv[2]);
MFEM_VERIFY(d_len > 1e-12, "...degenerate dip vector...");
const real_t d_inv = 1.0 / d_len;
dv[0] *= d_inv; dv[1] *= d_inv; dv[2] *= d_inv;
```

In exact arithmetic $|\mathbf{t}_1| = 1$ follows from $|\mathbf{t}_2| = 1$ and $\mathbf{t}_2 \perp \hat{\mathbf{n}}$. In floating point the strike normalize leaves $\mathbf{t}_2$ a few ULP off unit, and that error propagates into the $\mathbf{t}_2 \times \hat{\mathbf{n}}$ cross product, so $|\mathbf{t}_1|$ drifts ~5

ULP from 1. Downstream `BuildRotation` / `BuildRotationInverse` and the per-side flux consumers *assume* a strictly orthonormal frame, so dip is re-normalized defensively (added in v9.1.0, plan §3.4 H-V91-B1).

---

6 `NormalNeedsFlipToCanonical` – the orientation rule

```
static bool NormalNeedsFlipToCanonical(const Vector &n_raw,
                                       const Vector &ref_normal, int dim)
{
   const real_t nl = n_raw.Norml2();                                    // (1)
   real_t dot = 0.0;
   for (int d = 0; d < dim; d++) { dot += n_raw(d) * ref_normal(d); }   // (2)
   constexpr real_t kRefNormalDegenerateRelTol = 1e-6;
   if (nl > 0.0 && std::abs(dot) > kRefNormalDegenerateRelTol * nl)     // (3)
   {
      return (dot < 0.0);                                               // (4)
   }
   int kmax = 0;                                                        // (5)
   for (int d = 1; d < dim; d++)
   { if (std::abs(n_raw(d)) > std::abs(n_raw(kmax))) { kmax = d; } }
   return (n_raw(kmax) < 0.0);                                          // (6)
}
```

6.1 What single question does this answer?

Exactly one yes/no question: **"Must I negate `n_raw` to put it into canonical orientation?"** The caller does nothing more than

```
if (NormalNeedsFlipToCanonical(n_raw, ref_normal, dim)) { n_raw.Neg(); }
// ... n_raw is now guaranteed to be the canonical arrow.
```

So the first thing to pin down is what *canonical orientation* means. The physical normal of a face is a **line** — two opposite arrows, $+\mathbf{n}$ and $-\mathbf{n}$ — and `CalcOrtho` may hand us *either one* depending on context (§3). "Canonical" is a **rule that deterministically picks one of those two arrows**, so that whichever arrow we are handed, after the optional flip we are always holding the *same* one. The function returns `true` precisely when the arrow we were handed is the *wrong* one and must be negated.

The rule has two cases — the generic one (lines 2–4) and a fallback for a degenerate geometry (lines 5–6).

6.2 Generic case: align with `ref_normal` – lines (2)-(4)

Canonical $\equiv$ "the arrow with $\mathbf{n} \cdot$ `ref_normal` > 0." Compute `dot = n_raw · ref_normal`. If it is already positive, `n_raw` is the canonical arrow $\rightarrow$ `return false`. If negative, `n_raw` is the opposite

arrow → `return true` (caller flips it).

Why this is a **pure function of the line**: negating the input negates `dot`, which flips the answer, which means **both arrows end up canonical after the flip**. Concretely, take `ref_normal = (0,-1,0)` and a fault whose normal lies along y — the two sides of the face are handed opposite arrows:

handed `n_raw`	`dot`	returns	after the flip
$(0,+1,0)$	-1	`true`	$(0,-1,0)$
$(0,-1,0)$	$+1$	`false`	$(0,-1,0)$

Both inputs land on $(0,-1,0)$. That agreement — the same canonical arrow no matter which way `CalcOrtho` pointed — **is the entire purpose of the function.**

6.3 The guard – line (3): when is `dot` trustworthy?

Write `dot = nl · cos`θ, where θ is the angle between the normal and `ref_normal` (here `nl = |n_raw|` and `|ref_normal| = 1`). Then `|dot|/nl = |cos`θ`|`, so the guard `|dot| > 1e-6 · nl` is literally the test

$$|\cos\theta| > 10^{-6} \quad \Longleftrightarrow \quad \text{the normal is more than } \sim 10^{-6} \text{ rad away from } \perp \text{ to } \texttt{ref\_normal}.$$

Why that threshold works: the rounding error in `dot` is about $\varepsilon_{\text{machine}} \cdot nl \approx 10^{-16}\, nl$ — ten orders of magnitude *below* the `1e-6·nl` guard. So **above** the guard the sign of `dot` is real geometry and every rank computes it identically; **below** it the true `cos`θ is buried in rounding noise and `sign(dot)` is effectively a coin flip. The guard is exactly the dividing line between "trust the sign" and "don't." (The tolerance is **relative** to `nl`, so it is mesh-scale-independent — no hardcoded absolute length.)

6.4 Degenerate case: largest component positive – lines (5)-(6)

When the normal is $\approx\perp$ to `ref_normal` (`|cos`θ`|` $\leq$ `1e-6`) we **cannot** use `dot`: two ranks could read opposite signs of a noise-level number and disagree, which would break the very agreement the function exists to provide. This is not hypothetical — a ~N–S-striking SAFS segment has a normal pointing roughly E–W, $\mathbf{n} \approx (\pm 1, 0, 0)$, which is perpendicular to `ref_normal = (0,-1,0)`, so `dot` $\approx$ `0`.

So in this band we switch to a rule that ignores `ref_normal` entirely and looks only at the arrow's own components: **canonical** $\equiv$ **"the largest-magnitude component is positive."** Find the index `kmax` of the biggest $|n_{\text{raw}}(d)|$; the arrow is canonical iff $n_{\text{raw}}(\texttt{kmax}) > 0$.

This rule is simultaneously **line-invariant** and **noise-free**:

- **Line-invariant** — $|n_{\text{raw}}(d)|$ is identical for $+\mathbf{n}$ and $-\mathbf{n}$, so both pick the same `kmax`, and exactly one of the two arrows has that component positive. (Ties break to the lowest index, also the same for both.)
- **Noise-free** — `kmax` is the *dominant* component, far from zero, so its sign is genuine geometry, not rounding.

Same E–W example, `ref_normal = (0,-1,0)`:

handed `n_raw`	`kmax`	$n($`kmax`$)$	returns	after the flip
$(+1,0,0)$	0	$+1$	`false`	$(+1,0,0)$
$(-1,0,0)$	0	-1	`true`	$(+1,0,0)$

Both land on $(+1,0,0)$ — agreement again, this time without ever touching the unreliable `dot`.

6.5 Byte-exactness for planar faults

A planar TPV/BP5 fault has $\hat{\mathbf{n}} = \pm$ `ref_normal`, i.e. $|\cos\theta| = 1$ and `|dot| = nl`. The guard in (3) is therefore *always* satisfied, the degenerate branch is **never** entered, and the result is the plain `sign(dot)`. Every planar regression is bit-for-bit unchanged — the fallback only ever changes behaviour on a non-planar fault.

6.6 Two consumers, one rule

The method is called from exactly two places, so the *fault-frame sign* and the $\pm$*-side label* can never disagree about which way is canonical:

1. `ComputeOrientedFrame` step (2) $\rightarrow$ the stored `sign_flipped`.
2. The wave-operator `elem1_on_plus` side test (`wave_operator.inl:472, 546`), which projects the element-centroid offset onto the **canonical face normal** rather than the fixed `ref_normal`. (This is the Part A blow-up fix; see `PLAN_shared_fault_reconcile_fix`.)

The takeaway. `CalcOrtho` gives us a normal *line* but not a consistent *arrow*. This function collapses the two possible arrows to one — by aligning with `ref_normal` where that is meaningful, and by "largest component positive" where `ref_normal` is too close to perpendicular to be reliable. Either way both arrows map to the same canonical one, so two ranks sharing a face always agree.

---

7 Public API and the setup lifecycle

`FaultBasis` is a member of `ElasticityDomainOperator` (`elasticity_operator.hpp:488`) and of `WaveOperator` (`wave_operator.hpp:814`). It is filled **once**, during operator construction, in a strict four-step order (`elasticity_operator_setup.inl:739–786`):

```
1. Compute(interior_faces, ref_normal, up)              // centroid frame, interior
2. AppendSharedFaces(shared_faces, ref_normal, up)    // [parallel] centroid frame, shared
3. ComputeQPBasis(interior_faces, ..., face_ir)         // per-QP frame, interior
4. ComputeQPBasisShared(shared_faces, ..., face_ir,   // [parallel] per-QP frame, shared
                        interior_face_count)
```

The order is **mandatory**:

- `Compute` runs first — it sets `dim_` and `num_faces_` and sizes `basis_`. Everything else writes *into* that array.

- `AppendSharedFaces` must precede `ComputeQPBasisShared`: it grows `num_faces_` and resizes `basis_` to make room for the shared faces' QP data.
- `ComputeQPBasis*` only fills the `qp_data` member of an already-sized `FaultBasisData`.

7.1 The face-index mapping (critical in parallel)

`FaultBasis` stores interior and shared faces in one flat array:

$$\underbrace{[\,0,\ N_{\text{int}}\,)}_{\text{interior}} \qquad \underbrace{[\,N_{\text{int}},\ N_{\text{int}} + N_{\text{shared}}\,)}_{\text{shared}}$$

That offset is why `ComputeQPBasisShared` takes `interior_face_count`, and why consumers index shared faces as `fault_interior_faces_.Size() + sf_idx` (e.g. `wave_operator.inl:2136, 3069`). Forget the offset and you read the wrong face's frame.

7.2 Setup methods

Method	Role
`Compute(mesh, faces, ref_normal, up)`	Centroid frame per interior face. `CalcOrtho` at the face centre → `ComputeOrientedFrame`. `MFEM_VERIFY(nl > 0)`.
`ComputeQPBasis(mesh, faces, ref_normal, up, ir)`	Per-QP frame per interior face. Loops the quadrature rule, `CalcOrtho` per QP. Must follow `Compute`.
`AppendSharedFaces<MeshType>(mesh, shared, ref_normal, up)`	`[parallel]` Centroid frame for shared faces via `GetSharedFaceTransformations`; appends to `basis_`. Guarded by `#ifdef MFEM_USE_MPI`.
`ComputeQPBasisShared<PMeshType>(mesh, shared, ..., ir, interior_count)`	`[parallel]` Per-QP frame for shared faces, written at index `interior_count + i`.

`AppendSharedFaces` and `ComputeQPBasisShared` are **templated on the mesh type** so the same source serves serial and parallel builds; the bodies are fully `#ifdef MFEM_USE_MPI`-guarded.

7.3 Consumer methods (the projections)

```
void   ProjectTraction(int fi, const real_t *T_global, real_t *tau_local) const;
void   EmbedSlip      (int fi, const real_t *slip_local, real_t *du_global) const;
void   EmbedSlipQP    (int fi, int q, const real_t *slip_local, real_t *du_global) const;
real_t NormalStress   (int fi, const real_t *T_global) const;
```

- **`ProjectTraction`** — $\tau_1 = \mathbf{T} \cdot \mathbf{t}_1$ (dip), $\tau_2 = \mathbf{T} \cdot \mathbf{t}_2$ (strike, 3D only). Bare dot products with the *stored signed* tangents.
- **`NormalStress`** — $\sigma_n = -\mathbf{T} \cdot \hat{\mathbf{n}}$. The minus sign makes $\sigma_n > 0$ in **compression** (geology convention).
- **`EmbedSlip`** — $\Delta\mathbf{u} = s_1\mathbf{t}_1 + s_2\mathbf{t}_2$ using the centroid tangents.

- **`EmbedSlipQP`** — same, but with the per-QP tangents `qp_data[q]`. If `qp_data` is empty or `q` is out of range it **falls back to `EmbedSlip`** (centroid). This silent fallback is deliberate, but means a missing `ComputeQPBasis` call degrades to centroid accuracy without an error.

Why no sign factor is needed. Slip is *embedded* with the same stored tangent that traction is *projected* onto. The sign baked into that tangent therefore cancels in the slip→traction→friction→slip round trip — embedding and projecting are mutually consistent by construction. This is the payoff of the signed-vector contract (§3).

---

8 Call graph (critical path)

```
ElasticityDomainOperator::SetupFaultBasis()           // construction, once
  ├─→ FaultBasis::Compute()
  │     └─→ FaultBasis::ComputeOrientedFrame()          // per interior face centroid
  ├─→ FaultBasis::AppendSharedFaces()         [parallel]
  │     └─→ FaultBasis::ComputeOrientedFrame()          // per shared face centroid
  ├─→ FaultBasis::ComputeQPBasis()
  │     └─→ FaultBasis::ComputeOrientedFrame()          // per interior QP
  └─→ FaultBasis::ComputeQPBasisShared()      [parallel]
        └─→ FaultBasis::ComputeOrientedFrame()          // per shared QP

ElasticityDomainOperator::GetFaultTraction()          // every RHS evaluation
  ├─→ FaultBasis::GetBasis()
  ├─→ FaultBasis::ProjectTraction()                     // → dip/strike traction
  └─→ FaultBasis::NormalStress()                        // → σ_n

ElasticityDomainOperator::AssembleSlipRHS()           // every RHS evaluation
  ├─→ FaultBasis::EmbedSlipQP()  (→ EmbedSlip fallback)
  └─→ FaultBasis::EmbedSlip()

WaveOperator ctor (± side label)                       // dynamic rupture
  └─→ FaultBasis::NormalNeedsFlipToCanonical()       // canonical normal for side test

WaveOperator::Compute*FaceFluxRHS()                    // dynamic flux assembly
  └─→ FaultBasis::GetBasis()  (+ qpd.sign_flipped → rebuild canonical frame)
```

The static helpers `ComputeOrientedFrame` and `NormalNeedsFlipToCanonical` are the only methods exercised directly by unit tests (`test_fault_basis_dip_strike_symmetry`, `test_fault_basis_qp_orthonormality`, `test_fault_basis`).

---

9 Conventions

- **Sign baked in.** Stored vectors carry the `sign_flipped` negation; consumers never apply an extra sign. `sign_flipped` itself is informational for the vectors, but load-bearing where the flux reconstructs the canonical frame from it.
- **`real_t`** — MFEM's float/double typedef; the class is precision-agnostic.
- **Static + pure** — `ComputeOrientedFrame` and `NormalNeedsFlipToCanonical` hold no state, which is what makes the per-sample unit tests possible. They were promoted to `public` in v9.1.0 for exactly that reason (R-003), with a trailing `private:` block (R-008) so later members do not silently inherit public visibility.
- **Runtime invariants via `MFEM_VERIFY`** — zero-length normal (`nl > 0`), `up` collinear with the normal (`|up × n| > 1e-12`), degenerate dip (`|dip| > 1e-12`). These abort with a message rather than producing a silently wrong frame.
- **Parameters, not constants** — `ref_normal` and `up` are passed in. The class hardcodes no geometry; the chosen values live at the call sites.
- **MPI isolation** — every shared-face path is behind `#ifdef MFEM_USE_MPI` and templated on the mesh type, so serial builds compile the same header unchanged.

---

10 Gotchas

- **Do not re-apply a sign.** The vectors are already signed. A $\pm$ `sign` factor on top of `ProjectTraction/EmbedSlip` is a double flip — the canonical positive-feedback blow-up this whole subsystem is engineered to avoid (`CLAUDE.md`: *"Sign error → positive feedback → blowup"*).
- **The two-sided global sign flip is correct.** Frames on opposite sides of a shared face differ by an overall sign. The DG face normal flips identically, so it cancels in assembly (proof: code comment lines 492–502; `test_serial_parallel_displacement_match < 1e-12`). Do not "fix" it.
- **`n_raw` is modified in place.** `ComputeOrientedFrame` takes `Vector &n_raw` and flips/normalizes it. Do not reuse the vector afterward expecting the raw value.
- **`nl` is the *pre-flip* length.** It is captured before `Neg()`. Normalizing by it after the flip is correct only because negation preserves length — don't reorder those two lines.
- **The shared-face index offset.** Shared faces live at `interior_count + sf_idx` in `basis_`. Indexing a shared face as if it were interior reads the wrong frame.
- **`EmbedSlipQP` silently falls back to centroid.** If `ComputeQPBasis` was never called (empty `qp_data`), `EmbedSlipQP` degrades to centroid accuracy with no warning. On a curved fault that is a silent loss of per-QP fidelity.
- **The degenerate-band fallback is only reachable on a non-planar fault.** On planar TPV/BP5 faults `|dot| = nl`, so the `sign(dot)` branch is always taken and the fallback never runs. It activates only where a fault segment is $\approx\perp$ to `ref_normal` — so a bug in the largest-component tie-break would be invisible to every planar regression.
- **2D dip is not separately normalized.** It inherits unit length from the unit normal; the explicit R-003 normalize is 3D-only.

---

11 Open questions

- **Depth-sign wording in `CLAUDE.md`.** Two statements coexist: the fault-local-frame section gives `can_t1 = (0,0,-1)` ("down-dip") with "$Z < 0$ is depth," while the sign-conventions list says "Dip direction $(0, 0, +1)$ = downward into earth." The *code* produces the canonical dip $(0, 0, -1)$ on the vertical fault (worked above). Worth a one-line reconciliation on which prose statement is authoritative — the code itself is unambiguous.
- **Is the degenerate fallback exercised in production?** It is proven on the tilted-fault unit fixture, but whether the real SAFS mesh has fault segments inside the `|dot|` $\leq$ `1e-6·nl` band (vs merely tilted) has not been measured here. If never hit in production, it is pure insurance; if hit, its largest-component tie-break is load-bearing and deserves a dedicated regression on the production mesh.